Feature folder ke ander Motion_features.py

```
class MotionAnalyzer:
    def __init__(self):
        self.prev_nose = None

    def is_head_stable(self, landmarks):
        nose = (landmarks.landmark[1].x, landmarks.landmark[1].y)
        stable = True
        if self.prev_nose:
            dx = abs(nose[0] - self.prev_nose[0])
            dy = abs(nose[1] - self.prev_nose[1])
            stable = dx < 0.01 and dy < 0.01
        self.prev_nose = nose
        return stable
```

Health folder ke ander

attention_level.py

```
class AttentionAnalyzer:
    def __init__(self):
        pass
    def update(self, landmarks, head_stable):
        #  --- Simple: if head is stable → attention good ----
        return "High" if head_stable else "Low"
```

Eye_blink.py

```
import numpy as np
import math

def euclidean(p1, p2):
    return math.dist(p1, p2)

def eye_aspect_ratio(landmarks, eye_indices, frame_shape):
    h, w = frame_shape[:2]    # unpack properly
```

```
    points = [(int(landmarks.landmark[i].x *
w),int(landmarks.landmark[i].y * h))
            for i in eye_indices]

    # ---- EAR formula -----
    A = euclidean(points[1], points[5])
    B = euclidean(points[2], points[4])
    C = euclidean(points[0], points[3])

    if C == 0:
        return 0.0
    ear = (A + B) / (2.0 * C)
    return ear
class BlinkDetector:
    def __init__(self):
        self.left_indices = [33, 160, 158, 133, 153, 144]
        self.right_indices = [362, 385, 387, 263, 373, 380]

        self.last_left_ear = 0.0
        self.last_right_ear = 0.0

        self.left_closed = False
        self.right_closed = False

        self.EAR_THRESHOLD = 0.21

        self.left_count = 0
        self.right_count = 0

    def update(self, landmarks, frame_shape):
        left_ear = eye_aspect_ratio(landmarks, self.left_indices,
frame_shape)
        right_ear = eye_aspect_ratio(landmarks, self.right_indices,
frame_shape)

            # ---- Latest EAR values store karo ----
        self.last_left_ear = left_ear
        self.last_right_ear = right_ear
```

```
        # ---- LEFT EYE -----
        if left_ear < self.EAR_THRESHOLD:
            if not self.left_closed:
                self.left_closed = True
        else:
            if self.left_closed:
                self.left_count += 1
                self.left_closed = False

        # --- RIGHT EYE ----
        if right_ear < self.EAR_THRESHOLD:
            if not self.right_closed:
                self.right_closed = True
        else:
            if self.right_closed:
                self.right_count += 1
                self.right_closed = False
        return {
            "left": self.left_count,
            "right": self.right_count,
            "both": min(self.left_count, self.right_count)
        }
```

Eye_strain.py

```
class EyeStrainDetector:
    def __init__(self, ear_limit=0.25):
        self.ear_limit = ear_limit
        self.strain_time = 0.0

    def update(self, avg_ear, elapsed_time):
        # --- Option 2: time-based accumulation ----
        if avg_ear < self.ear_limit:
            self.strain_time += elapsed_time
        return round(self.strain_time, 2)
```

Facial_Color.py

```
import cv2
import numpy as np
class FacialColorAnalyzer:
    def analyze(self, roi):
        avg_color = cv2.mean(roi)[:3]
        if avg_color[1] > 100:
            return "Normal"
        else:
            return "Pale"
```

Fatigue.py

```
class FatigueDetector:
    def __init__(self, blink_threshold=20):
        self.blink_threshold = blink_threshold

    def update(self, avg_ear):
        if avg_ear < 0.21:
            return "High"
        return "Normal"
```

Head_stability.py

```
import math
class HeadStabilityAnalyzer:
    def __init__(self):
        self.prev_point = None

    def update(self, point):
        if self.prev_point is None:
            self.prev_point = point
            return True

        dist = math.dist(self.prev_point, point)
        self.prev_point = point
```

```
        return dist < 5
```

Heart_rate.py

```
import numpy as np
class HeartRateAnalyzer:
    def __init__(self):
        self.prev_mean = 0

    def update(self, green_channel):
        # ----- Simple rPPG demo: mean pixel intensity ------
        mean_val = np.mean(green_channel)
        bpm = int((mean_val / 255) * 60 + 60)  # dummy formula
        return bpm
```

Posture.py

```
import math
class PostureAnalyzer:
    def analyze(self, nose, chin):
        dy = chin[1] - nose[1]
        dx = chin[0] - nose[0]
        angle = math.degrees(math.atan2(dy, dx))
        if abs(angle) < 15:
            return "Good"
        return "Bad"
```

Resp_rate.py

```
class RespirationAnalyzer:
    def __init__(self):
        self.prev_y = None
        self.breath_count = 0

    def update(self, nose_y):
        if self.prev_y:
            if abs(nose_y - self.prev_y) > 2:
```

```
                self.breath_count += 1
        self.prev_y = nose_y
        return self.breath_count
```

Infrastructure folde ke ander

[Camera.py]

```
import cv2
class Camera:
    def __init__(self, src=0, width=640, height=480):
        self.cap = cv2.VideoCapture(src)
        self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, width)
        self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, height)

    def get_frame(self):
        ret, frame = self.cap.read()
        if not ret:
            return None
        return frame

    def release(self):
        self.cap.release()
```

Utils folder ke ander

logger.py

```
def log(msg):
    print(f"[LOG] {msg}")
```

Vision folder ke ander
Face_detector.py

```
import cv2
import mediapipe as mp
class FaceDetector:
```

```
    def __init__(self):
        self.mp_face = mp.solutions.face_detection
        self.detector =
self.mp_face_detection.FaceDetection(model_selection=0,min_detection_confi
dence=0.5)

    def detect(self, frame):
        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = self.detector.process(rgb)
        faces = []
        if results.detections:
            h, w, _ = frame.shape
            for detection in results.detections:
                bbox = detection.location_data.relative_bounding_box
                x = int(bbox.xmin * w)
                y = int(bbox.ymin * h)
                bw = int(bbox.width * w)
                bh = int(bbox.height * h)
                faces.append((x, y, bw, bh))

        return faces
```

landmarks.py

```
import mediapipe as mp
import cv2
class FaceLandmarks:
    def __init__(self):
        self.mp_face_mesh = mp.solutions.face_mesh
        self.mesh =
self.mp_face_mesh.FaceMesh(static_image_mode=False,max_num_faces=1,

refine_landmarks=True,min_detection_confidence=0.5,min_tracking_confidence
=0.5)

    def extract(self, frame):
        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = self.mesh.process(rgb)
        if results.multi_face_landmarks:
```

```
            return results.multi_face_landmarks[0]
        return None
```

[app.py](app.py)

```
import cv2
import mediapipe as mp
import numpy as np
import time

from src.health.eye_blink import BlinkDetector
from src.health.eye_strain import EyeStrainDetector
from src.health.fatigue import FatigueDetector
from src.health.attention_level import AttentionAnalyzer
from src.features.motion_features import MotionAnalyzer
from src.health.heart_rate import HeartRateAnalyzer
from src.health.resp_rate import RespirationAnalyzer
from src.health.facial_color import FacialColorAnalyzer
from src.health.posture import PostureAnalyzer

# ============================
# ---- MediaPipe FaceMesh -----
# ============================
mp_face_mesh = mp.solutions.face_mesh
face_mesh = mp_face_mesh.FaceMesh(max_num_faces=1,refine_landmarks=True,

min_detection_confidence=0.5,min_tracking_confidence=0.5)
# =========================
# ------ Camera --------
# =========================
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    print("❌ Camera not accessible")
    exit()
# ============================
# ---- Initialize Detectors ---
# ============================
detectors = {
```

```
    "blink": BlinkDetector(),
    "eye_strain": EyeStrainDetector(),
    "fatigue": FatigueDetector(),
    "motion": MotionAnalyzer(),
    "attention": AttentionAnalyzer(),
    "heart_rate": HeartRateAnalyzer(),
    "respiration": RespirationAnalyzer(),
    "facial_color": FacialColorAnalyzer(),
    "posture": PostureAnalyzer()
}

print("✅ Health Monitoring System Started")

# ============================
# ---- Helper: Draw Label ---
# ============================
def draw_label(frame, text, x, y, color=(255, 255, 255)):
    cv2.putText(frame, text, (x, y), cv2.FONT_HERSHEY_SIMPLEX,0.55, color,
1, cv2.LINE_AA)

# ================================
# --- Helper: Update Metrics ----
# ================================
def update_metrics(frame, landmarks):
    h, w, _ = frame.shape
    metrics = {}

    # ---- Eye Blink -----
    metrics["blink"] = detectors["blink"].update(landmarks, frame.shape)

    # ----- Eye Strain (Option 2: time-based) -----
    avg_ear = (detectors["blink"].last_left_ear +
detectors["blink"].last_right_ear)/2
    metrics["eye_strain"] = detectors["eye_strain"].update(avg_ear, 1/30)

    # ---- Fatigue ----
    metrics["fatigue"] = detectors["fatigue"].update(avg_ear)

    # ---- Motion & Head Stability ----
    head_stable = detectors["motion"].is_head_stable(landmarks)
```

```
    # ---- Attention----
    metrics["attention"] = detectors["attention"].update(landmarks,
head_stable)

    # ---- Phase 3: Heart Rate & Facial Color ----
    x_min, x_max = int(0.3*w), int(0.7*w)
    y_min, y_max = int(0.35*h), int(0.6*h)
    roi = frame[y_min:y_max, x_min:x_max]
    if roi.size > 0:
        metrics["heart_rate"] = detectors["heart_rate"].update(roi[:, :,
1])
        metrics["facial_color"] = detectors["facial_color"].analyze(roi)
    else:
        metrics["heart_rate"] = metrics["facial_color"] = "--"

    # ---- Respiration ----
    nose_y = int(landmarks.landmark[1].y * h)
    metrics["respiration"] = detectors["respiration"].update(nose_y)

    # ---- Posture ----
    nose = (int(landmarks.landmark[1].x*w),
int(landmarks.landmark[1].y*h))
    chin = (int(landmarks.landmark[152].x*w),
int(landmarks.landmark[152].y*h))
    metrics["posture"] = detectors["posture"].analyze(nose, chin)

    return metrics

# ========================
# ----- Main Loop -------
# ========================
while True:
    ret, frame = cap.read()
    if not ret:
        break
    frame = cv2.flip(frame, 1)
    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    result = face_mesh.process(rgb_frame)
```

```
    #  --- metrics ----
    metrics =
{"blink":"--","eye_strain":"--","fatigue":"--","attention":"--",

"heart_rate":"--","respiration":"--","facial_color":"--","posture":"--"}

    if result.multi_face_landmarks:
        landmarks = result.multi_face_landmarks[0]
        metrics = update_metrics(frame, landmarks)

    # ========================
    # ---- UI Overlay ------
    # ========================
    overlay = frame.copy()
    cv2.rectangle(overlay, (0,0), (380,360), (0,0,0), -1)
    cv2.addWeighted(overlay, 0.5, frame, 0.5, 0, frame)

    # Labels
    draw_label(frame, "FACE HEALTH MONITOR", 10, 25, (0,255,255))
    draw_label(frame, f"Blink Count : {metrics['blink']}", 10, 60)
    draw_label(frame, f"Eye Strain  : {metrics['eye_strain']}", 10, 85)
    draw_label(frame, f"Fatigue     : {metrics['fatigue']}", 10, 110)
    draw_label(frame, f"Attention   : {metrics['attention']}", 10, 135)
    draw_label(frame, f"Heart Rate  : {metrics['heart_rate']} BPM", 10,
165)
    draw_label(frame, f"Resp Rate   : {metrics['respiration']}", 10, 190)
    draw_label(frame, f"Face Color  : {metrics['facial_color']}", 10, 215)
    draw_label(frame, f"Posture     : {metrics['posture']}", 10, 240)
    draw_label(frame, "Non-medical | Approximate Results", 10, 330,
(0,0,255))

    cv2.imshow("Face Health Monitoring System", frame)
    if cv2.waitKey(1) & 0xFF == 27:  # ESC
        break
# ========================
# --- Cleanup ------
# ========================
cap.release()
cv2.destroyAllWindows()
print("🛑 System Stopped")
```